

Assessment 2

Use dataset from Assessment 1 to train a machine learning model on historical records then apply it to a simulated real-time stream. This assessment supports unit learning outcomes 3,4 and 5.

AI acknowledgement: Chat-GPT was used in this task to debug code throughout and get a better understanding of Kafka environment, especially when it came to setting up Docker containers and making sure all were communicating with each other.

Additional acknowledgement: a lot of the below code was structured around weekly activity tasks, with some code directly copied and then altered to accommodate dataset and task required.

Before Part A

Data is split into training dataset, 70% of records, to train and evaluate the ML model and remaining 30% will be used for streaming dataset and will be replayed through Kafka in Part B.

```
In [21]: from pyspark import SparkConf
from pyspark.sql import SparkSession

#setup a spark session and load dataset
master = "local[*]"
app_name = "Uber Business Query DF"
spark_conf = SparkConf().setMaster(master).setAppName(app_name)

spark = SparkSession.builder.config(conf=spark_conf).getOrCreate()

sc = spark.sparkContext
```

```
In [22]: df = spark.read.csv("ncr_ride_bookings.csv",header=True)
```

```
In [23]: train_df1, stream_df = df.randomSplit([0.7,0.3],seed = 42)

#to ensure a 70/30 split of data
print(train_df1.count())
print(train_df1.count()/150000)
print(stream_df.count())
print(stream_df.count()/150000)

#repeat of assessment one to ensure data type is correct
from pyspark.sql.functions import col,concat_ws
from pyspark.sql.types import DateType

train_df1 = train_df1.replace("null", None)
```

```
float_col = ["Avg VTAT", "Avg CTAT", "Ride Distance", "Driver Ratings"]
train_df1 = train_df1.select([col(c).cast("float") if c in float_col

#remove all entries that do not have a driver rating number, which i
#this is done after the initial split to reflect what real-time dat
train_df = train_df1.dropna(subset=["Driver Ratings"])
print(train_df.count())
```

```
105209
0.7013933333333333
44791
0.2986066666666667
65201
```

Part A - The ML Pipeline

1. The ML task selection

The ideal machine learning algorithm for the original business case of trying to predict the driver rating of their ride based on the record features would be regression. This is because the ratings are numerical in value, continuous after some data cleaning, and do have meaningful variance. The dataset included will only include records that do have a driver rating, which is much lower than the original dataset but still large enough to create a model with meaningful results.

2. Feature Engineering

For this dataset, there are four general types of data that are included - categorical, binary, numerical and free text. Free text is separated as it requires further work to categorise this data, such as feature extraction and feature transformation. Binary is also removed, as it can easily be converted to full numerical with NULL converted to 0. Booking ID and Customer ID are removed, as they are unique and identifying features that would not provide any predictable value.

```
In [24]: train_df.show(5)
```



```
train_df = (train_df.withColumn("Hour", hour("Time").cast("string"))  
.withColumn("Month", month("Date").cast("string")))  
train_df.show(5)
```



```
In [26]: train_df = train_df.drop("Date", "Time")
train_df.show(5)
```


The date and time were converted into hour, day of the week and month as this will be more relevant to the final model. Even though they are numerical values representing specific features, they will be treated at categories as they represent a specific time of day or year rather than a numerical number.

```
In [27]: #seperate numerically relevant columns from category columns as well
categoryInputCols = ["Booking Status", "Vehicle Type", "Pickup Location", "Drop Location"]
binaryInputCols = ["Cancelled Rides by Customer", "Cancelled Rides by Driver", "Incomplete Rides"]
freetextInputCols = ["Reason for cancelling by Customer", "Driver Cancellation Reason"]
numericInputCols = ["Avg VTAT", "Avg CTAT", "Booking Value", "Ride Distance"]
numericOutputCol = "Driver Ratings"

numericalCols = [numericOutputCol]+numericInputCols
```

```
In [28]: train_df.select(binaryInputCols).printSchema()
```

```
root
 |-- Cancelled Rides by Customer: string (nullable = true)
 |-- Cancelled Rides by Driver: string (nullable = true)
 |-- Incomplete Rides: string (nullable = true)
```

```
In [29]: #convert binary input cols into full numerical with Null = 0.0. since
from pyspark.sql.functions import when

for c in binaryInputCols:
    train_df = train_df.withColumn(c,when(col(c).isNull(), 0.0).otherwise(1.0))
train_df.show()
```

```
+-----+-----+-----+-----+-----+
|      Booking ID|Booking Status|      Customer ID|Vehicle Type|      P
| pickup Location|      Drop Location|Avg VTAT|Avg CTAT|Cancelled Ride
| s by Customer|Reason for cancelling by Customer|Cancelled Rides by D
| river|Driver Cancellation Reason|Incomplete Rides|Incomplete Rides R
| eason|Booking Value|Ride Distance|Driver Ratings|Customer Rating|Pay
| ment Method|Hour|DayOfWeek|Month|
+-----+-----+-----+-----+-----+
|      "CNR9927107"|      Completed|"CID8187312"|      Go Mini|
| Janakpuri|      Karkarduma|      6.8|      44.0|
| 0.0|
| NULL|      0.0|      NULL|      261.0|
| 25.03|      4.2|      4.5|      Debit Card|      5|      2|
| 1|
|      "CNR7840176"|      Completed|"CID2800770"|      Go Sedan|
```

Samaypur Badli	Old Gurgaon	14.7	23.1		
0.0		NULL		0.0	
NULL	0.0		NULL	344.0	
16.76	3.8	4.9	UPI	8	2
1					
""CNR7743565""	Completed	""CID2074867""		Auto	
Pitampura	Narsinghpur	11.5	16.0		
0.0		NULL		0.0	
NULL	0.0		NULL	654.0	
40.48	4.1	4.1	Debit Card	9	2
1					
""CNR6288298""	Completed	""CID4484069""		Bike	
Vatika Chowk	Jor Bagh	6.7	31.7		
0.0		NULL		0.0	
NULL	0.0		NULL	398.0	
20.54	5.0	4.5	Uber Wallet	10	2
1					
""CNR9442408""	Completed	""CID9819315""		Auto	
Nehru Place	Manesar	2.5	32.0		
0.0		NULL		0.0	
NULL	0.0		NULL	125.0	
24.21	4.2	4.4	Cash	11	2
1					
""CNR8261535""	Completed	""CID4421149""		Auto	
Mehrauli	Lajpat Nagar	13.3	19.7		
0.0		NULL		0.0	
NULL	0.0		NULL	489.0	
47.96	4.3	3.1	Cash	13	2
1					
""CNR8666573""	Completed	""CID3448187""		Go Mini	
ITO	Azadpur	2.9	35.0		
0.0		NULL		0.0	
NULL	0.0		NULL	285.0	
7.95	4.2	3.7	UPI	15	2
1					
""CNR4251366""	Completed	""CID4436460""		Uber XL	
New Colony	Saidulajab	2.8	26.0		
0.0		NULL		0.0	
NULL	0.0		NULL	434.0	
23.23	4.3	4.2	Credit Card	16	2
1					
""CNR2869355""	Completed	""CID7652461""		Go Mini Farid	
abad Sector 15	Dwarka Mor	3.5	17.4		
0.0		NULL		0.0	
NULL	0.0		NULL	347.0	
31.88	4.4	4.4	Uber Wallet	16	2
1					
""CNR4289111""	Completed	""CID8225531""		Bike	
Hauz Rani	Sarojini Nagar	14.3	18.2		
0.0		NULL		0.0	
NULL	0.0		NULL	423.0	
42.5	3.7	5.0	Cash	17	2
1					
""CNR1774511""	Completed	""CID7880605""		Go Sedan Centr	
al Secretariat	Panchsheel Park	13.8	35.7		
0.0		NULL		0.0	

NULL	0.0	NULL	304.0	
41.48	4.3	4.5	Uber Wallet	17 2
1				
"CNR2689650"	Completed	"CID5638785"	eBike	
Peeragarhi	Dwarka Sector 21	4.4	44.6	
0.0		NULL	0.0	
NULL	0.0	NULL	272.0	
41.8	4.5	4.9	Cash	17 2
1				
"CNR3027089"	Completed	"CID8425693"	Auto	
Chhatarpur	Gurgaon Railway S...	10.6	18.8	
0.0		NULL	0.0	
NULL	0.0	NULL	951.0	
37.97	3.4	4.2	UPI	18 2
1				
"CNR2994610"	Completed	"CID3927468"	Auto	
Patel Chowk	Vinobapuri	4.1	41.7	
0.0		NULL	0.0	
NULL	0.0	NULL	548.0	
43.91	4.2	3.9	Debit Card	18 2
1				
"CNR7778425"	Completed	"CID3524152"	Auto	
Kadarpur	Paschim Vihar	7.8	40.1	
0.0		NULL	0.0	
NULL	0.0	NULL	207.0	
18.69	3.8	4.3	Debit Card	18 2
1				
"CNR3914591"	Completed	"CID3217781"	Bike	
Pragati Maidan	Vasant Kunj	2.2	21.7	
0.0		NULL	0.0	
NULL	0.0	NULL	147.0	
31.04	4.2	4.6	UPI	19 2
1				
"CNR7294075"	Completed	"CID6392882"	Bike	
Qutub Minar	Kanhaiya Nagar	11.7	18.1	
0.0		NULL	0.0	
NULL	0.0	NULL	739.0	
36.6	4.2	4.2	UPI	20 2
1				
"CNR6786441"	Completed	"CID7659776"	Bike	
Chirag Delhi	Sarojini Nagar	4.6	30.6	
0.0		NULL	0.0	
NULL	0.0	NULL	636.0	
31.53	3.7	3.0	Uber Wallet	20 2
1				
"CNR8613340"	Completed	"CID7944365"	Bike	
Lal Quila	Gurgaon Railway S...	5.4	42.8	
0.0		NULL	0.0	
NULL	0.0	NULL	314.0	
22.92	4.3	5.0	UPI	20 2
1				
"CNR5992770"	Completed	"CID8289400"	Go Sedan	
Khandsa	Old Gurgaon	7.2	42.0	
0.0		NULL	0.0	
NULL	0.0	NULL	300.0	
15.97	4.4	4.6	Cash	21 2

```
1|
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
only showing top 20 rows
```

2a. Investigation into Payment method

As part of feature extraction, I needed to investigate payment method and impact on the specific part of the dataset I was looking at. Due to that, I went back and investigated the impact of removing all null Driver Ratings from above and reassessed what columns I needed within my model. After the below investigation, the concluding assumption is that only rides that are complete allow the customer to give the driver a rating. A rating cannot be given if the booking is incomplete or cancelled, however if it is incomplete a payment is still taken and recorded. Based on this investigation, it was determined that Booking Status and any information relating to cancellation or incomplete would have no influence on the predictive model to determine Driver Ratings. Therefore these will be removed from the features used in model.

```
In [30]: #Review null values in category columns to see if data is missing o

#only looking at the non-altered categories
null_categoryInputCols = ["Booking Status", "Vehicle Type", "Pickup

#converting null values for streaming and original dataset
df_null = df.replace("null", None)
stream_null = stream_df.replace("null", None)

from pyspark.sql.functions import col, sum
print("dataset before the removing of null driver ratings")
train_df1.select([sum(col(c).isNull().cast("int")).alias(c) for c in
print("dataset after the removing of null driver ratings")
train_df.select([sum(col(c).isNull().cast("int")).alias(c) for c in
print("streaming dataset with null")
stream_null.select([sum(col(c).isNull().cast("int")).alias(c) for c in
print("total dataset with null")
df_null.select([sum(col(c).isNull().cast("int")).alias(c) for c in n
```

dataset before the removing of null driver ratings

```
+-----+-----+-----+-----+-----+
-----+
|Booking Status|Vehicle Type|Pickup Location|Drop Location|Payment M
ethod|
+-----+-----+-----+-----+-----+
-----+
|              0|              0|              0|              0|
33638|
+-----+-----+-----+-----+-----+
-----+
```

dataset after the removing of null driver ratings

```
+-----+-----+-----+-----+-----+
-----+
|Booking Status|Vehicle Type|Pickup Location|Drop Location|Payment M
ethod|
+-----+-----+-----+-----+-----+
-----+
|              0|              0|              0|              0|
0|
+-----+-----+-----+-----+-----+
-----+
```

streaming dataset with null

```
+-----+-----+-----+-----+-----+
-----+
|Booking Status|Vehicle Type|Pickup Location|Drop Location|Payment M
ethod|
+-----+-----+-----+-----+-----+
-----+
|              0|              0|              0|              0|
14362|
+-----+-----+-----+-----+-----+
-----+
```

total dataset with null

```
+-----+-----+-----+-----+-----+
-----+
|Booking Status|Vehicle Type|Pickup Location|Drop Location|Payment M
ethod|
+-----+-----+-----+-----+-----+
-----+
|              0|              0|              0|              0|
48000|
+-----+-----+-----+-----+-----+
-----+
```

```
In [31]: from pyspark.sql.functions import col, count

df_null.groupBy("Booking Status").agg(
    sum(col("Driver Ratings").isNull().cast("int")).alias("Missing
    sum(col("Payment Method").isNull().cast("int")).alias("Missing
    count("*").alias("Total")
).show()
```

	Booking Status	Missing Driver Rating	Missing Payment Method	Total
3000	Completed	0	0	9
0500	No Driver Found	10500	10500	1
7000	Cancelled by Driver	27000	27000	2
0500	Cancelled by Cust...	10500	10500	1
9000	Incomplete	9000	0	

2b. Vector Assesmbler

```
In [32]: #final set of columns that will be used in vector assessmbler and m
#seperate numerically relevant columns from category columns as wel
categoryInputCols = ["Vehicle Type", "Pickup Location", "Drop Locat
numericInputCols = ["Avg VTAT", "Avg CTAT", "Booking Value", "Ride
numericOutputCol = "Driver Ratings"

numericalCols = [numericOutputCol]+numericInputCols
```

```
In [33]: from pyspark.ml.feature import StringIndexer

# Define output column names
outputCols = [f'{x}_index' for x in categoryInputCols]

print(categoryInputCols)
print(outputCols)

# Create index values for categorical input features
inputIndexer = StringIndexer(inputCols=categoryInputCols,outputCols

# Call the fit and transform() method to get the encoded results
train_df_indexed = inputIndexer.fit(train_df).transform(train_df)
# Display the output, only the output columns
train_df_indexed.select(outputCols).show(10)
```



```

model = encoder.fit(train_df_indexed)
train_df_encoded = model.transform(train_df_indexed)
# Display the output columns
train_df_encoded.select(outputCols_OHE).show(5)

```

```

+-----+-----+-----+-----+
|Vehicle Type_vec|Pickup Location_vec|Drop Location_vec|Payment Meth
od_vec|      Hour_vec|DayOfWeek_vec|      Month_vec|
+-----+-----+-----+-----+
|  (7, [1], [1.0])|  (176, [104], [1.0])|(176, [118], [1.0])|      (5, [4],
[1.0])|(24, [18], [1.0])|(7, [5], [1.0])|(12, [0], [1.0])|
|  (7, [2], [1.0])|  (176, [16], [1.0])|(176, [120], [1.0])|      (5, [0],
[1.0])|(24, [12], [1.0])|(7, [5], [1.0])|(12, [0], [1.0])|
|  (7, [0], [1.0])|  (176, [95], [1.0])|(176, [23], [1.0])|      (5, [4],
[1.0])|(24, [8], [1.0])|(7, [5], [1.0])|(12, [0], [1.0])|
|  (7, [3], [1.0])|  (176, [169], [1.0])|(176, [67], [1.0])|      (5, [2],
[1.0])|(24, [3], [1.0])|(7, [5], [1.0])|(12, [0], [1.0])|
|  (7, [0], [1.0])|  (176, [96], [1.0])|(176, [70], [1.0])|      (5, [1],
[1.0])|(24, [6], [1.0])|(7, [5], [1.0])|(12, [0], [1.0])|
+-----+-----+-----+-----+

```

only showing top 5 rows

The second part requires converting the numerical data of each of the features into a sparse vector that stores non-zero values and their indices. For each vector the first value is the number of types available in that specific feature, so for Vehicle Type the first value for all vectors is 7, then the indices of where the value is within the all values, and finally the value itself. Since each vector is a single, the value for all entries will be 1.0.

```

In [35]: from pyspark.ml.feature import VectorAssembler

# inputCols are all the encoded columns from OHE plus numerical col
assemblerInputs = outputCols_OHE + numericInputCols

# Combine all input features into a single feature vector
assembler = VectorAssembler(inputCols=assemblerInputs, outputCol="fe

# Transform the encoded dataframe
train_df_final = assembler.transform(train_df_encoded)

# Display features and numerical output
train_df_final.select("features", numericOutputCol).show(truncate=F

```

```

+-----+-----+
|features
|Driver Ratings|
+-----+-----+
| (412, [1, 111, 301, 363, 382, 393, 395, 407, 408, 409, 410, 411], [1.0, 1.0, 1.0,

```

1.0,1.0,1.0,1.0,6.800000190734863,44.0,261.0,25.030000686645508,4.
 5)) |4.2 |
 |(412,[2,23,303,359,376,393,395,407,408,409,410,411],[1.0,1.0,1.0,1.
 0,1.0,1.0,1.0,14.699999809265137,23.100000381469727,344.0,16.7600002
 28881836,4.900000095367432])) |3.8 |
 |(412,[0,102,206,363,372,393,395,407,408,409,410,411],[1.0,1.0,1.0,
 1.0,1.0,1.0,1.0,11.5,16.0,654.0,40.47999954223633,4.09999990463256
 8)) |4.1 |
 |(412,[3,176,250,361,367,393,395,407,408,409,410,411],[1.0,1.0,1.0,
 1.0,1.0,1.0,1.0,6.699999809265137,31.700000762939453,398.0,20.540000
 915527344,4.5)) |5.0 |
 |(412,[0,103,253,360,370,393,395,407,408,409,410,411],[1.0,1.0,1.0,
 1.0,1.0,1.0,1.0,2.5,32.0,125.0,24.209999084472656,4.40000009536743
 2)) |4.2 |
 |(412,[0,12,211,360,377,393,395,407,408,409,410,411],[1.0,1.0,1.0,1.
 0,1.0,1.0,1.0,13.300000190734863,19.700000762939453,489.0,47.9599990
 84472656,3.0999999046325684))] |4.3 |
 |(412,[1,55,296,359,373,393,395,407,408,409,410,411],[1.0,1.0,1.0,1.
 0,1.0,1.0,1.0,2.9000000953674316,35.0,285.0,7.949999809265137,3.7000
 00047683716))] |4.2 |
 |(412,[6,94,313,362,368,393,395,407,408,409,410,411],[1.0,1.0,1.0,1.
 0,1.0,1.0,1.0,2.799999952316284,26.0,434.0,23.229999542236328,4.1999
 99809265137))] |4.3 |
 |(412,[1,129,189,361,368,393,395,407,408,409,410,411],[1.0,1.0,1.0,
 1.0,1.0,1.0,1.0,3.5,17.399999618530273,347.0,31.8799991607666,4.4000
 00095367432))] |4.4 |
 |(412,[3,153,239,360,365,393,395,407,408,409,410,411],[1.0,1.0,1.0,
 1.0,1.0,1.0,1.0,14.300000190734863,18.200000762939453,423.0,42.5,5.
 0))] |3.7 |
 |(412,[2,140,197,361,365,393,395,407,408,409,410,411],[1.0,1.0,1.0,
 1.0,1.0,1.0,1.0,13.800000190734863,35.70000076293945,304.0,41.479999
 54223633,4.5))] |4.3 |
 |(412,[5,44,323,360,365,393,395,407,408,409,410,411],[1.0,1.0,1.0,1.
 0,1.0,1.0,1.0,4.400000095367432,44.599998474121094,272.0,41.79999923
 706055,4.900000095367432))] |4.5 |
 |(412,[0,179,199,359,364,393,395,407,408,409,410,411],[1.0,1.0,1.0,
 1.0,1.0,1.0,1.0,10.600000381469727,18.799999237060547,951.0,37.97000
 1220703125,4.199999809265137))] |3.4 |
 |(412,[0,123,330,363,364,393,395,407,408,409,410,411],[1.0,1.0,1.0,
 1.0,1.0,1.0,1.0,4.099999904632568,41.70000076293945,548.0,43.9099998
 4741211,3.9000000953674316))] |4.2 |
 |(412,[0,134,227,363,364,393,395,407,408,409,410,411],[1.0,1.0,1.0,
 1.0,1.0,1.0,1.0,7.800000190734863,40.099998474121094,207.0,18.690000
 534057617,4.300000190734863))] |3.8 |
 |(412,[3,45,339,359,366,393,395,407,408,409,410,411],[1.0,1.0,1.0,1.
 0,1.0,1.0,1.0,2.200000047683716,21.700000762939453,147.0,31.04000091
 5527344,4.599999904632568))] |4.2 |
 |(412,[3,172,294,359,369,393,395,407,408,409,410,411],[1.0,1.0,1.0,
 1.0,1.0,1.0,1.0,11.699999809265137,18.100000381469727,739.0,36.59999
 8474121094,4.199999809265137))] |4.2 |
 |(412,[3,177,239,361,369,393,395,407,408,409,410,411],[1.0,1.0,1.0,
 1.0,1.0,1.0,1.0,4.599999904632568,30.600000381469727,636.0,31.530000
 686645508,3.0))] |3.7 |
 |(412,[3,161,199,359,369,393,395,407,408,409,410,411],[1.0,1.0,1.0,
 1.0,1.0,1.0,1.0,5.400000095367432,42.79999923706055,314.0,22.9200000
 76293945,5.0))] |4.3 |

```
| (412, [2, 7, 303, 360, 371, 393, 395, 407, 408, 409, 410, 411], [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 7.199999809265137, 42.0, 300.0, 15.970000267028809, 4.599999904632568]) |4.4 |
+-----+
+-----+
+-----+
only showing top 20 rows
```

3/4. Model Comparison and Evaluation

```
In [36]: #train_df is further split into 70/30 to allow models to be trained
train, test = train_df_final.randomSplit([0.7, 0.3], seed = 2020)
print("Training Dataset Count: " + str(train.count()))
print("Test Dataset Count: " + str(test.count()))

train.show(5)
```

```
Training Dataset Count: 45555
Test Dataset Count: 19646
```

Booking ID	Booking Status	Customer ID	Vehicle Type	Pick up Location	Drop Location	Avg VTAT	Avg CTAT	Cancelled Rides by Customer	Reason for cancelling by Customer	Cancelled Rides by Driver	Driver Cancellation Reason	Incomplete Rides	Incomplete Rides Reason	Booking Value	Ride Distance	Driver Ratings	Customer Rating	Payment Method	Hour	DayOfWeek	Month	Vehicle Type_index	Pickup Location_index	Drop Location_index	Payment Method_index	Hour_index	DayOfWeek_index	Month_index	Vehicle Type_vec	Pickup Location_vec	Drop Location_vec	Payment Method_vec	Hour_vec	DayOfWeek_vec	Month_vec											
""CNR1003564""	Completed	""CID2527345""	Premier Sedan	Jhilmil	Punjabi Bagh	2.0	19.6	0.0		0.0		0.0		0.0		0.0		389.0	45.52	3.2						3.6	Credit Card	15		6	1	4.0	124.0	12.0	3.0	9.0	3.0	0.0	(7,[4],[1.0])	(176,[124],[1.0])	(176,[12],[1.0])	(5,[3],[1.0])	(24,[9],[1.0])	(7,[3],[1.0])	(12,[0],[1.0])	(412,[4,131,195,3...]
""CNR1003981""	Completed	""CID6905297""	Auto	Dwarka Mor	Shastri Nagar	10.8	30.9	0.0		0.0		0.0		0.0		0.0		207.0	4.83	4.2						3.8	UPI	6		7	7	0.0	52.0	20.0	0.0	16.0	0.0	3.0	(7,[0],[1.0])	(176,[52],[1.0])	(176,[20],[1.0])	(5,[0],[1.0])	(24,[16],[1.0])	(7,[0],[1.0])	(12,[3],[1.0])	(412,[0,59,203,35...]
""CNR1003988""	Completed	""CID7154196""	Bike	Ashok Vihar	Vinobapuri	11.3	23.6	0.0		0.0		0.0		0.0		0.0		397.0	24.72	5.0						4.6	Uber Wallet	6		4	1	3.0	101.0	147.0	2.0	16.0	4.0	0.0	(7,[3],[1.0])	(176,[101],[1.0])	(176,[147],[1.0])	(5,[2],[1.0])	(24,[16],[1.0])	(7,[4],[1.0])	(12,[0],[1.0])	(412,[3,108,330,3...]
""CNR1005546""	Completed	""CID9703583""	Premier Sedan	Govindpuri	Ghitorni Village	14.8	19.9	0.0		0.0		0.0		0.0		0.0		242.0	9.66	4.2						4.6	Credit Card	23		3	10	4.0	157.0	163.0	3.0	17.0	1.0	2.0	(7,[4],[1.0])	(176,[						

26/10/04 00:19:50 WARN Instrumentation: [0cef5c7a] regParam is zero, which might cause numerical instability and overfitting.
 26/10/04 00:19:51 WARN Instrumentation: [0cef5c7a] Cholesky solver failed due to singular covariance matrix. Retrying with Quasi-Newton solver.

Driver Ratings	prediction
4.4	4.149727015335577
4.6	4.290834459055684
4.3	4.265141597746981
4.2	4.216547416156796
4.1	4.212980762453869
4.2	4.248500940760183
4.6	4.229237735088293
4.7	4.24497800920897
4.5	4.258383070388338
3.2	4.201292830464332
4.1	4.183222754261615
4.4	4.249449889676875
4.9	4.18905258800651
4.7	4.199448337906711
4.5	4.323200864801352
4.2	4.276633147727592
5.0	4.233456181080399
3.1	4.243305908534888
4.1	4.296325497918988
3.8	4.209563365401953

only showing top 20 rows

```
In [21]: evaluate_regression(lrPredictions,numericOutputCol)
```

RMSE: 0.439
 R²: -0.009
 MAE: 0.333

```
Out[21]: (0.43935041911630623, -0.00866754124798308, 0.3332435890324029)
```

```
In [22]: #introducing regularisation to control the fit of the model
lr_l2 = LinearRegression(featuresCol="features",labelCol=numericOut
lrl2_model = lr_l2.fit(train)
lrl2_Predictions = lrl2_model.transform(test)
evaluate_regression(lrl2_Predictions,numericOutputCol)
```

RMSE: 0.439
 R²: -0.006
 MAE: 0.332

```
Out[22]: (0.4387037829129314, -0.005700611268282918, 0.332432740220418)
```

```
In [23]: print(len(train.select("features").first()["features"]))
```

412

3/4a. Removal of location feature

Based on the large number of features within the df_final dataset, mainly due to the large number of locations possible in the pickup and drop-off location, a separate dataset will be created without those columns to see if this could create a better suited model.

```
In [40]: categoryInputCols_nolocation = ["Vehicle Type", "Payment Method", "Ho

# Define output column names
outputCols = [f'{x}_index' for x in categoryInputCols_nolocation]

# Create index values for categorical input features
inputIndexer1 = StringIndexer(inputCols=categoryInputCols_nolocation)
train_df_indexed_noloc = inputIndexer1.fit(train_df).transform(train_df)

# Input columns are the indexed categorical input columns
inputCols_OHE = outputCols
outputCols_OHE = [f'{x}_vec' for x in categoryInputCols_nolocation]

# Define OneHotEncoder with the appropriate columns
encoder1 = OneHotEncoder(inputCols=inputCols_OHE, outputCols=outputCols_OHE)
model1 = encoder1.fit(train_df_indexed_noloc)
train_df_encoded1 = model1.transform(train_df_indexed_noloc)

# inputCols are all the encoded columns from OHE plus numerical columns
assemblerInputs1 = outputCols_OHE + numericInputCols
assembler1 = VectorAssembler(inputCols=assemblerInputs1, outputCol="features")
train_df_noloc = assembler1.transform(train_df_encoded1)

# number of features within the new dataset
print(len(train_df_noloc.select("features").first()["features"]))

train_noloc, test_noloc = train_df_noloc.randomSplit([0.7, 0.3], seed=1)
print("No location training Dataset Count: " + str(train_noloc.count()))
print("No location test Dataset Count: " + str(test_noloc.count()))

60
No location training Dataset Count: 45555
No location test Dataset Count: 19646
```

```
In [41]: #repeat of linear regression with no location dataset
lr2 = LinearRegression(featuresCol="features", labelCol="numericOutputCol")
lr2_model = lr2.fit(train_noloc)
lr2_Predictions = lr2_model.transform(test_noloc)
evaluate_regression(lr2_Predictions, numericOutputCol)
```

```
26/10/04 00:20:26 WARN Instrumentation: [399390ce] regParam is zero,
which might cause numerical instability and overfitting.
26/10/04 00:20:27 WARN Instrumentation: [399390ce] Cholesky solver failed
due to singular covariance matrix. Retrying with Quasi-Newton solver.
```

```
RMSE: 0.438
R²: -0.001
MAE: 0.331
```

```
Out[41]: (0.43765859970665405, -0.0009142871363132521, 0.33136585385446443)
```

```
In [42]: #repeat with regularisation parameter
lr2_l2 = LinearRegression(featuresCol="features",labelCol=numericOu
lr2_l2_model = lr2_l2.fit(train_noloc)
lr2_l2_Predictions = lr2_l2_model.transform(test_noloc)
evaluate_regression(lr2_l2_Predictions,numericOutputCol)
```

RMSE: 0.438
R²: -0.001
MAE: 0.331

```
Out[42]: (0.4376585104360168, -0.000913878816919178, 0.3313658350899033)
```

```
In [43]: #trying a decision tree model to see if that improves the fitting
from pyspark.ml.regression import DecisionTreeRegressor

dt = DecisionTreeRegressor(featuresCol="features",labelCol=numericO

# Train model with no location data
dt_model = dt.fit(train_noloc)
dt_predictions = dt_model.transform(test_noloc)

dt_results = evaluate_regression(dt_predictions,numericOutputCol)
```

RMSE: 0.438
R²: -0.004
MAE: 0.332

```
In [44]: dt1 = DecisionTreeRegressor(featuresCol="features",labelCol=numeric

# Train model with no location data
dt1_model = dt1.fit(train_noloc)
dt1_predictions = dt1_model.transform(test_noloc)

dt1_results = evaluate_regression(dt1_predictions,numericOutputCol)
```

RMSE: 0.438
R²: -0.004
MAE: 0.332

```
In [45]: dt2 = DecisionTreeRegressor(featuresCol="features",labelCol=numeric

# Train model with no location data
dt2_model = dt2.fit(train_noloc)
dt2_predictions = dt2_model.transform(test_noloc)

dt2_results = evaluate_regression(dt2_predictions,numericOutputCol)
```

RMSE: 0.465
R²: -0.132
MAE: 0.357

```
In [46]: from pyspark.ml.regression import RandomForestRegressor

rf = RandomForestRegressor(featuresCol="features",labelCol=numericO

rf_model = rf.fit(train_noloc)
rf_predictions = rf_model.transform(test_noloc)
```

```
rf_results = evaluate_regression(rf_predictions,numericOutputCol)
```

RMSE: 0.438

R²: -0.001

MAE: 0.331

```
In [31]: rf1 = RandomForestRegressor(featuresCol="features",labelCol=numeric
      featureSubsetStrategy="sqrt",seed=42)

      rf1_model = rf1.fit(train_noloc)
      rf1_predictions = rf1_model.transform(test_noloc)

      rf1_results = evaluate_regression(rf1_predictions,numericOutputCol)
```

```
26/10/03 09:49:04 WARN DAGScheduler: Broadcasting large task binary
with size 1128.5 KiB
26/10/03 09:49:04 WARN DAGScheduler: Broadcasting large task binary
with size 1759.5 KiB
26/10/03 09:49:04 WARN DAGScheduler: Broadcasting large task binary
with size 2.6 MiB
26/10/03 09:49:05 WARN DAGScheduler: Broadcasting large task binary
with size 3.7 MiB
26/10/03 09:49:05 WARN DAGScheduler: Broadcasting large task binary
with size 5.2 MiB
26/10/03 09:49:06 WARN DAGScheduler: Broadcasting large task binary
with size 7.1 MiB
26/10/03 09:49:07 WARN DAGScheduler: Broadcasting large task binary
with size 1119.0 KiB
26/10/03 09:49:07 WARN DAGScheduler: Broadcasting large task binary
with size 9.3 MiB
26/10/03 09:49:07 WARN DAGScheduler: Broadcasting large task binary
with size 1345.1 KiB
26/10/03 09:49:08 WARN DAGScheduler: Broadcasting large task binary
with size 12.0 MiB
26/10/03 09:49:09 WARN DAGScheduler: Broadcasting large task binary
with size 1581.0 KiB
26/10/03 09:49:09 WARN DAGScheduler: Broadcasting large task binary
with size 14.9 MiB
26/10/03 09:49:10 WARN DAGScheduler: Broadcasting large task binary
with size 1772.9 KiB
```

RMSE: 0.438

R²: -0.004

MAE: 0.332

```
In [32]: rf2 = RandomForestRegressor(featuresCol="features",labelCol=numeric
      featureSubsetStrategy="sqrt",seed=42)

      rf2_model = rf2.fit(train_noloc)
      rf2_predictions = rf2_model.transform(test_noloc)

      rf2_results = evaluate_regression(rf2_predictions,numericOutputCol)
```

```
26/10/03 09:49:13 WARN DAGScheduler: Broadcasting large task binary
with size 1524.7 KiB
26/10/03 09:49:14 WARN DAGScheduler: Broadcasting large task binary
with size 2.5 MiB
26/10/03 09:49:15 WARN DAGScheduler: Broadcasting large task binary
with size 4.0 MiB
26/10/03 09:49:16 WARN DAGScheduler: Broadcasting large task binary
with size 6.2 MiB
26/10/03 09:49:16 WARN DAGScheduler: Broadcasting large task binary
with size 1278.5 KiB
26/10/03 09:49:17 WARN DAGScheduler: Broadcasting large task binary
with size 9.2 MiB
26/10/03 09:49:18 WARN DAGScheduler: Broadcasting large task binary
with size 1750.3 KiB
```

```
RMSE: 0.438
R²: -0.001
MAE: 0.331
```

```
In [33]: rf3 = RandomForestRegressor(featuresCol="features",labelCol=numeric
      featureSubsetStrategy="sqrt",seed=42)

rf3_model = rf3.fit(train_noloc)
rf3_predictions = rf3_model.transform(test_noloc)

rf3_results = evaluate_regression(rf3_predictions,numericOutputCol)
```

```
RMSE: 0.438
R²: -0.000
MAE: 0.331
```

3/4b. Investigation into important features

As most of these models have very similar metrics and none showing a good fitting model, an investigation into the most influential features will be done. After, the dataset may be reduced further to have only these features included.

```
In [34]: #translating the features from the spare vector back into understand
metadata = train_df_noloc.schema["features"].metadata
attrs = metadata["ml_attr"]["attrs"]
feature_names = []

for attr_type in attrs:
    for attr in attrs[attr_type]:
        feature_names.append((attr["idx"], attr["name"]))

# Sort by feature index
feature_names = [name for idx, name in sorted(feature_names)]
print(feature_names)
```

```
['Vehicle Type_vec_Auto', 'Vehicle Type_vec_Go Mini', 'Vehicle Type_vec_Go Sedan', 'Vehicle Type_vec_Bike', 'Vehicle Type_vec_Premier Sedan', 'Vehicle Type_vec_eBike', 'Vehicle Type_vec_Uber XL', 'Payment Method_vec_UPI', 'Payment Method_vec_Cash', 'Payment Method_vec_Uber Wallet', 'Payment Method_vec_Credit Card', 'Payment Method_vec_Debit Card', 'Hour_vec_18', 'Hour_vec_17', 'Hour_vec_19', 'Hour_vec_10', 'Hour_vec_16', 'Hour_vec_20', 'Hour_vec_11', 'Hour_vec_21', 'Hour_vec_9', 'Hour_vec_15', 'Hour_vec_14', 'Hour_vec_12', 'Hour_vec_8', 'Hour_vec_13', 'Hour_vec_22', 'Hour_vec_7', 'Hour_vec_6', 'Hour_vec_23', 'Hour_vec_5', 'Hour_vec_0', 'Hour_vec_2', 'Hour_vec_3', 'Hour_vec_4', 'Hour_vec_1', 'DayOfWeek_vec_7', 'DayOfWeek_vec_3', 'DayOfWeek_vec_1', 'DayOfWeek_vec_6', 'DayOfWeek_vec_4', 'DayOfWeek_vec_2', 'DayOfWeek_vec_5', 'Month_vec_1', 'Month_vec_3', 'Month_vec_10', 'Month_vec_7', 'Month_vec_5', 'Month_vec_6', 'Month_vec_12', 'Month_vec_4', 'Month_vec_8', 'Month_vec_11', 'Month_vec_9', 'Month_vec_2', 'Avg VTAT', 'Avg CTAT', 'Booking Value', 'Ride Distance', 'Customer Rating']
```

```
In [35]: importances = rf_model.featureImportances.toArray()
         importance_data = [(feature_names[i], float(importances[i])) for i in range(feature_names.size)]
         importance_df = spark.createDataFrame(importance_data, ["Feature", "Importance"])

         # Sort from highest to lowest importance
         importance_df = importance_df.orderBy(col("Importance").desc())

         importance_df.show(20, truncate=False)
```

Feature	Importance
Avg VTAT	0.1150809131140762
Booking Value	0.11226058654551944
Customer Rating	0.0956754528244088
Ride Distance	0.0944531066054288
Avg CTAT	0.08743093245931786
DayOfWeek_vec_3	0.019640554150751915
Hour_vec_8	0.018855698970600434
Hour_vec_13	0.01745850993292213
Hour_vec_3	0.017145498760657532
Hour_vec_1	0.016821547777566045
Hour_vec_20	0.015916770035366356
Vehicle Type_vec_Go Mini	0.01582350283497823
Vehicle Type_vec_Bike	0.015768647104559042
DayOfWeek_vec_2	0.01393011317084144
Payment Method_vec_UPI	0.013160186536103203
Month_vec_2	0.01302238216224268
Payment Method_vec_Cash	0.013010355944037353
Month_vec_10	0.012692437126981454
Payment Method_vec_Credit Card	0.012620828194130759
Hour_vec_5	0.012420170279199044

only showing top 20 rows

Since it looks like the numerical features have highest importance, another test dataset with only numerical features potentially should be created.

However, there is an understanding that for the categorical features, they have been split into further features so their importance may be split over several different features. Therefore below, these split features will be aggregated together to show the overall importance compared back to the original features.

```
In [36]: from pyspark.sql.functions import split

#finding the original features based on the additional name _vec_u
importance_grouped = importance_df.withColumn("Original_Feature",
    when(col("Feature").contains("_vec_"),split(col("Feature"), "_v
#grouping and summing the importance values for the original featur
grouped_importance = (importance_grouped.groupBy("Original_Feature"
    .orderBy(col("Total_Importance").desc()))

grouped_importance.show(truncate=False)
```

Original_Feature	Total_Importance
Hour	0.20054581571751665
Avg VTAT	0.1150809131140762
Booking Value	0.11226058654551944
Customer Rating	0.0956754528244088
Ride Distance	0.0944531066054288
Avg CTAT	0.08743093245931786
Month	0.08601662799432805
Vehicle Type	0.0839229955218349
DayOfWeek	0.07066377000671546
Payment Method	0.053949799210853765

Reviewing the importance of difference features, the following have an importance over 9.4%: Hour, Avg VTAT, Booking Value, Customer Rating and Ride Distance. Acknowledging this is only importance shown for a general random forest model, it will be used as a basis for the final dataset and modelling as a decision tree type model makes sense for a ratings type feature. Grouping these numerical values into specific ranges then deciding if that would lead to a specific range of rating would be the best approach for this dataset. Further, these features make sense from a realistic point of view. Customers most important features would be down to price, time it takes for driver to reach the pickup location and time of day.

```
In [37]: final_categoryInputCols = ["Hour"]
final_numericInputCols = ["Avg VTAT", "Booking Value", "Ride Distan

#completing vector assembly for final category inputs

outputCols2 = [f'{x}_index' for x in final_categoryInputCols]
# Create index values for categorical input features
inputIndexer2 = StringIndexer(inputCols=final_categoryInputCols,out
```

```

train_df_indexed_final = inputIndexer2.fit(train_df).transform(trai

# Input columns are the indexed categorical input columns
inputCols_OHE2 = outputCols2
outputCols_OHE2 = [f'{x}_vec' for x in final_categoryInputCols]

#Define OneHotEncoder with the appropriate columns
encoder2 = OneHotEncoder(inputCols=inputCols_OHE2,outputCols=output
model2 = encoder2.fit(train_df_indexed_final)
train_df_encoded2 = model2.transform(train_df_indexed_final)

# inputCols are all the encoded columns from OHE plus numerical col
assemblerInputs2 = outputCols_OHE2 + final_numericInputCols
assembler2 = VectorAssembler(inputCols=assemblerInputs2,outputCol="
train_df_final= assembler2.transform(train_df_encoded2)

#number of features within the new dataset, should be 24+4=28
print(len(train_df_final.select("features").first()["features"]))

train_final, test_final = train_df_final.randomSplit([0.7, 0.3], se
print("Final training Dataset Count: " + str(train_final.count()))
print("Final test Dataset Count: " + str(test_final.count()))

```

28

Final training Dataset Count: 45555

Final test Dataset Count: 19646

In [38]: train_final.show(10)

```

+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+
|      Booking ID|Booking Status|      Customer ID| Vehicle Type| Pic
kup Location|   Drop Location|Avg VTAT|Avg CTAT|Cancelled Rides by C
ustomer|Reason for cancelling by Customer|Cancelled Rides by Driver|
Driver Cancellation Reason|Incomplete Rides|Incomplete Rides Reason|
Booking Value|Ride Distance|Driver Ratings|Customer Rating|Payment M
ethod|Hour|DayOfWeek|Month|Hour_index|      Hour_vec|      fe
atures|
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+
|""""CNR1003564""""|      Completed|""""CID2527345""""|Premier Sedan|
Jhilmil|      Punjabi Bagh|      2.0|      19.6|
0.0|
NULL|      0.0|      NULL|      389.0|
45.52|      3.2|      3.6|      Credit Card|      15|      6|
1|      9.0| (24, [9], [1.0])|(28, [9,24,25,26,2...|
|""""CNR1003981""""|      Completed|""""CID6905297""""|      Auto|

```

Dwarka Mor	Shastri Nagar	10.8	30.9		
0.0		NULL		0.0	
NULL	0.0		NULL	207.0	
4.83	4.2	3.8	UPI	6	7
7	16.0 (24,[16],[1.0]) (28,[16,24,25,26,...				
""CNR1003988""	Completed	""CID7154196""		Bike	
Ashok Vihar	Vinobapuri	11.3	23.6		
0.0		NULL		0.0	
NULL	0.0		NULL	397.0	
24.72	5.0	4.6	Uber Wallet	6	4
1	16.0 (24,[16],[1.0]) (28,[16,24,25,26,...				
""CNR1005546""	Completed	""CID9703583""	Premier Sedan		
Govindpuri	Ghitorni Village	14.8	19.9		
0.0		NULL		0.0	
NULL	0.0		NULL	242.0	
9.66	4.2	4.6	Credit Card	23	3
10	17.0 (24,[17],[1.0]) (28,[17,24,25,26,...				
""CNR1006399""	Completed	""CID8873292""		Auto	
Tilak Nagar	Rithala	12.7	40.1		
0.0		NULL		0.0	
NULL	0.0		NULL	134.0	
20.05	3.6	4.1	UPI	21	5
6	7.0 (24,[7],[1.0]) (28,[7,24,25,26,2...				
""CNR1008018""	Completed	""CID3811912""		Auto	Anan
d Vihar ISBT	Moti Nagar	13.4	20.0		
0.0		NULL		0.0	
NULL	0.0		NULL	237.0	
30.89	3.3	4.9	Credit Card	22	7
3	14.0 (24,[14],[1.0]) (28,[14,24,25,26,...				
""CNR1008586""	Completed	""CID1608645""		Go Sedan	
Vasant Kunj	Bahadurgarh	7.8	25.3		
0.0		NULL		0.0	
NULL	0.0		NULL	404.0	
18.74	3.6	4.3	Debit Card	19	1
8	2.0 (24,[2],[1.0]) (28,[2,24,25,26,2...				
""CNR1010802""	Completed	""CID6799671""		Auto	
Hauz Rani	Dwarka Mor	9.1	35.2		
0.0		NULL		0.0	
NULL	0.0		NULL	897.0	
13.67	4.3	3.3	UPI	23	1
11	17.0 (24,[17],[1.0]) (28,[17,24,25,26,...				
""CNR1013600""	Completed	""CID6223112""		eBike	Gre
ater Kailash	IIT Delhi	9.7	15.6		
0.0		NULL		0.0	
NULL	0.0		NULL	356.0	
14.9	4.3	3.4	UPI	7	7
12	15.0 (24,[15],[1.0]) (28,[15,24,25,26,...				
""CNR1013758""	Completed	""CID1888914""		Bike	
Dwarka Mor	Anand Vihar ISBT	5.2	18.5		
0.0		NULL		0.0	
NULL	0.0		NULL	838.0	
20.98	4.3	4.1	UPI	15	4
10	9.0 (24,[9],[1.0]) (28,[9,24,25,26,2...				
+	+	+	+	+	+
+	+	+	+	+	+
+	+	+	+	+	+

```

-----+-----+-----+
-----+-----+-----+
---+---+---+---+---+---+
-----+

```

only showing top 10 rows

In [39]: *#create three separate general models with final dataset to investi*

```

lr_final = LinearRegression(featuresCol = "features", labelCol=nu
lrModel_final = lr.fit(train_final)
lrPredictions_final = lrModel_final.transform(test_final)

print("linear regression")
lr_results_final = evaluate_regression(lrPredictions_final,numeric0

dt_final = DecisionTreeRegressor(featuresCol="features",labelCol=nu
dt_model_final = dt_final.fit(train_final)
dt_predictions_final = dt_model_final.transform(test_final)

print("decision tree")
dt_results_final = evaluate_regression(dt_predictions_final,numeric

rf_final = RandomForestRegressor(featuresCol="features",labelCol=nu
rf_model_final = rf.fit(train_final)
rf_predictions_final = rf_model_final.transform(test_final)

print("random forest")
rf_results_final = evaluate_regression(rf_predictions_final,numeric

```

```

26/10/03 09:49:24 WARN Instrumentation: [dc959905] regParam is zero,
which might cause numerical instability and overfitting.
26/10/03 09:49:25 WARN Instrumentation: [dc959905] Cholesky solver f
ailed due to singular covariance matrix. Retrying with Quasi-Newton
solver.

```

```

linear regression
RMSE: 0.438
R²:   -0.000
MAE:   0.331
decision tree
RMSE: 0.439
R²:   -0.006
MAE:   0.333
random forest
RMSE: 0.438
R²:   -0.001
MAE:   0.331

```

In [40]: *#regression parameter investigate with linear model*

```

lr_final_l2 = LinearRegression(featuresCol="features",labelCol=nu
lr_final_l2_model = lr_final_l2.fit(train_final)
lr_final_l2_Predictions = lr_final_l2_model.transform(test_final)
lr_l2_results_final = evaluate_regression(lr_final_l2_Predictions,n

```

```

RMSE: 0.438
R²:   -0.000
MAE:   0.331

```

```
In [41]: #decision tree with specific depth and node values
dt_final_1 = DecisionTreeRegressor(featuresCol="features", labelCol=
dt_final_1_model = dt_final_1.fit(train_final)
dt_final_1_predictions = dt_final_1_model.transform(test_final)
dt_final_1_results = evaluate_regression(dt_final_1_predictions, num
```

```
26/10/03 09:49:32 WARN DAGScheduler: Broadcasting large task binary
with size 1130.7 KiB
26/10/03 09:49:32 WARN DAGScheduler: Broadcasting large task binary
with size 1310.6 KiB
26/10/03 09:49:32 WARN DAGScheduler: Broadcasting large task binary
with size 1504.5 KiB
RMSE: 0.499
R2: -0.303
MAE: 0.385
```

```
In [42]: #random forest with specific numebr of trees, depth and nodes
rf_final_1 = RandomForestRegressor(featuresCol="features", labelCol=
featureSubsetStrategy="sqrt", seed=42)
rf_final_1_model = rf_final_1.fit(train_final)
rf_final_1_predictions = rf_final_1_model.transform(test_final)
rf_final_1_results = evaluate_regression(rf_final_1_predictions, num
```

```
26/10/03 09:49:35 WARN DAGScheduler: Broadcasting large task binary
with size 1024.1 KiB
26/10/03 09:49:35 WARN DAGScheduler: Broadcasting large task binary
with size 1433.5 KiB
26/10/03 09:49:35 WARN DAGScheduler: Broadcasting large task binary
with size 1955.3 KiB
26/10/03 09:49:36 WARN DAGScheduler: Broadcasting large task binary
with size 2.6 MiB
26/10/03 09:49:36 WARN DAGScheduler: Broadcasting large task binary
with size 3.3 MiB
26/10/03 09:49:37 WARN DAGScheduler: Broadcasting large task binary
with size 4.3 MiB
26/10/03 09:49:38 WARN DAGScheduler: Broadcasting large task binary
with size 5.4 MiB
26/10/03 09:49:39 WARN DAGScheduler: Broadcasting large task binary
with size 6.7 MiB
26/10/03 09:49:40 WARN DAGScheduler: Broadcasting large task binary
with size 8.2 MiB
RMSE: 0.438
R2: -0.002
MAE: 0.332
```

3/4c. Final conclusions on dataset

From the above models, it seems that reducing the dataset further didn't have any impact on the regression statistics. Based on this, reviewing the variance of Driver Ratings and the correlation to the numerical values is done to evaluate if the a ideal model could even be created from this dataset. Based on the values below, the Driver Ratings do have a good distribution but only between 3-5. The standard deviation of 0.436 matches the overall RMSE of

the various models above. Looking at the correlation for just the numerical inputs, there is basically no relation between any of these values and the Driver Rating. Similarly for the Hour, the average rating for each hour is basically the same, which highlights that summing importance for a feature seperated into 24 features isn't the best approach. Based on this investigation, it can be concluded that the data available cannot create a model that will predict the Driver Rating. This may highlight that additional factors that aren't recorded may be needed. For example, cleanliness of vechile or communication with driver may be more impactful on Driver Rating, which is a very human metric and could be greatly influenced by many things.

```
In [43]: train_final.groupBy(numericOutputCol).count().orderBy(numericOutput
```

```
+-----+-----+
|Driver Ratings|count|
+-----+-----+
|          3.0|  349|
|          3.1|  729|
|          3.2|  759|
|          3.3|  727|
|          3.4|  752|
|          3.5|  334|
|          3.6|  971|
|          3.7| 1841|
|          3.8| 1873|
|          3.9| 1908|
|          4.0|   972|
|          4.1| 3449|
|          4.2| 6769|
|          4.3| 6943|
|          4.4| 3483|
|          4.5| 2206|
|          4.6| 4615|
|          4.7| 2306|
|          4.8| 1147|
|          4.9| 2262|
+-----+-----+
only showing top 20 rows
```

```
In [44]: train_final.select(numericOutputCol).describe().show()
```

```
+-----+-----+
|summary|  Driver Ratings|
+-----+-----+
|  count|          45555|
|   mean|4.2311250067505615|
| stddev|0.4363564461287325|
|    min|             3.0|
|    max|             5.0|
+-----+-----+
```

```
In [45]: numericInputCols = ["Avg VTAT", "Avg CTAT", "Booking Value", "Ride
```

```

for feature in numericInputCols:

    correlation = train_df_final.stat.corr(feature,"Driver Ratings")

    print(f"{feature}: {correlation:.4f}")

```

Avg VTAT: -0.0027
 Avg CTAT: 0.0055
 Booking Value: 0.0015
 Ride Distance: -0.0016
 Customer Rating: 0.0019

In [46]: `from pyspark.sql.functions import avg, count, round`

```

train_df.groupBy("Hour").agg(
    round(
        avg("Driver Ratings"),
        3
    ).alias("Mean_Driver_Rating"),

    count("*").alias("Count")
).orderBy("Hour").show(24)

```

Hour	Mean_Driver_Rating	Count
0	4.258	611
1	4.208	581
10	4.224	4263
11	4.227	3664
12	4.24	3017
13	4.221	2362
14	4.228	3032
15	4.233	3527
16	4.228	4190
17	4.231	4823
18	4.221	5362
19	4.232	4786
2	4.234	603
20	4.248	4188
21	4.235	3585
22	4.237	2361
23	4.225	1227
3	4.218	593
4	4.241	586
5	4.246	1189
6	4.226	1833
7	4.232	2337
8	4.237	2921
9	4.235	3560

3/4d. Model Evaluation Table

The final evaluation table will be created from the six regression models

created using the 28 feature dataset, even though it was concluded none of these models are ideal based on the data available. Based on the below metrics, the model chosen for Part B investigation is 'Decision Tree Default'.

```
In [47]: import builtins
model_results = [
    ("Linear Regression",builtins.round(lr_results_final[0],3),builtins.round(lr_results_final[1],3),builtins.round(lr_results_final[2],3)),
    ("Linear Regression L2 Regression",builtins.round(lr_l2_results_final[0],3),builtins.round(lr_l2_results_final[1],3),builtins.round(lr_l2_results_final[2],3)),
    ("Decision Tree Default",builtins.round(dt_results_final[0],3),builtins.round(dt_results_final[1],3),builtins.round(dt_results_final[2],3)),
    ("Decision Tree D-20,N-3",builtins.round(dt_final_1_results[0],3),builtins.round(dt_final_1_results[1],3),builtins.round(dt_final_1_results[2],3)),
    ("Random Forest Default",builtins.round(rf_results_final[0],3),builtins.round(rf_results_final[1],3),builtins.round(rf_results_final[2],3)),
    ("Random Forest T-100,D-15,N-8",builtins.round(rf_final_1_results[0],3),builtins.round(rf_final_1_results[1],3),builtins.round(rf_final_1_results[2],3))
]

results_df = spark.createDataFrame(model_results,["Model", "RMSE", "R2", "MAE"])
results_df.show(truncate=False)
```

Model	RMSE	R2	MAE
Linear Regression	0.438	-0.0	0.331
Linear Regression L2 Regression	0.438	-0.0	0.331
Decision Tree Default	0.439	-0.006	0.333
Decision Tree D-20,N-3	0.499	-0.303	0.385
Random Forest Default	0.438	-0.001	0.331
Random Forest T-100,D-15,N-8	0.438	-0.002	0.332

5. Model Persistence

```
In [ ]: #saving best model
dt_model_final.write().overwrite().save("models/dt_model_final")
```

```
In [ ]: #dedicated notebook cell reloading saved model and confirming the schema
from pyspark.ml.regression import DecisionTreeRegressionModel

dt_model_loaded = DecisionTreeRegressionModel.load("models/dt_model_final")

#sample of test dataset
sample = test_final.limit(10)

# Predictions from original and loaded model
original_predictions = dt_model_final.transform(sample)
loaded_predictions = dt_model_loaded.transform(sample)

# Compare schemas
print("Schemas match:",original_predictions.schema == loaded_predictions.schema)
```

Schemas match: True

Part B - Streaming, Kafka and reflection

Part B has four components: a Kafka producer, a Spark Structured Streaming consumer, a windowed aggregation and an embedded reflection

1-4. Streaming data simulation

You will simulate a real-time data stream by replaying records from the held-out streaming subset through Kafka. Each batch represents a group of newly arriving events. The Spark consumer ingests these events, applies the persisted ML model and generates predictions in near real-time. The objective is to demonstrate streaming data processing, not collect live data from an external system.

1a. Creation of preprocessing pipeline

Note: the kernel in the jupyter notebook should be restarted here, as a separate Spark session is used for Part B

```
In [48]: #change location to match docker container location and make the fo
import os

os.chdir("/home/jovyan/work")
print("Working directory:", os.getcwd())
```

Working directory: /home/jovyan/work

```
In [49]: #first create pipelinemodel that can be used to complete the proces
from pyspark.ml import Pipeline
from pyspark.ml.feature import StringIndexer, OneHotEncoder, Vector

# Final model inputs
final_categoryInputCols = ["Hour"]
final_numericInputCols = ["Avg VTAT", "Booking Value", "Ride Distance"]

# Stage 1: Index categorical variable
outputCols2 = [f"{x}_index" for x in final_categoryInputCols]
inputIndexer2 = StringIndexer(inputCols=final_categoryInputCols, outputCols=outputCols2)

# Stage 2: One-hot encode indexed variable
inputCols_OHE2 = outputCols2
outputCols_OHE2 = [f"{x}_vec" for x in final_categoryInputCols]
encoder2 = OneHotEncoder(inputCols=inputCols_OHE2, outputCols=outputCols_OHE2)

# Stage 3: Assemble model features
assemblerInputs2 = (outputCols_OHE2 + final_numericInputCols)
assembler2 = VectorAssembler(inputCols=assemblerInputs2, outputCol="features")

# Build pipeline
stages = [inputIndexer2, encoder2, assembler2]

pipeline = Pipeline(stages=stages)

pipelineModel = pipeline.fit(train_df)
```



```

-----+-----+-----+-----+-----+-----+
--+-+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+
+-+--+-----+-----+-----+-----+-----+-----+
-+-+
only showing top 5 rows

```

```
In [ ]: #saving pipeline model so it can be called later
pipelineModel.write().overwrite().save("models/preprocessing_pipeli
```

1b. Spark structured streaming consumer

```
In [2]: from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = (SparkSession.builder.appName("Ride Rating Streaming Consum
"org.apache.spark:spark-sql-kafka-0-10_2.13:4.2.0").getOrCreate()
spark.sparkContext.setLogLevel("WARN")

print("Spark:", spark.version)
print("Scala:", spark.sparkContext._jvm.scala.util.Properties.version
```

```

WARNING: Using incubator modules: jdk.incubator.vector
:: loading settings :: url = jar:file:/opt/conda/lib/python3.11/site
-packages/pyspark/jars/ivy-2.5.3.jar!/org/apache/ivy/core/settings/ivysettings.xml
Ivy Default Cache set to: /home/jovyan/.ivy2.5.2/cache
The jars for the packages stored in: /home/jovyan/.ivy2.5.2/jars
org.apache.spark#spark-sql-kafka-0-10_2.13 added as a dependency
:: resolving dependencies :: org.apache.spark#spark-submit-parent-84
959c4f-c557-4147-85b2-6a7bac9c1f88;1.0
   confs: [default]
   found org.apache.spark#spark-sql-kafka-0-10_2.13;4.2.0 in ce
ntral
   found org.apache.spark#spark-token-provider-kafka-0-10_2.13;
4.2.0 in central
   found org.apache.kafka#kafka-clients;3.9.2 in central
   found at.yawk.lz4#lz4-java;1.11.0 in central
   found org.xerial.snappy#snappy-java;1.1.10.8 in central
   found org.slf4j#slf4j-api;2.0.17 in central
   found org.apache.hadoop#hadoop-client-runtime;3.5.0 in centr
al
   found org.apache.hadoop#hadoop-client-api;3.5.0 in central
   found com.google.code.findbugs#jsr305;3.0.0 in central
   found org.scala-lang.modules#scala-parallel-collections_2.1
3;1.2.0 in central
   found org.apache.commons#commons-pool2;2.13.1 in central
:: resolution report :: resolve 157ms :: artifacts dl 4ms
   :: modules in use:
   at.yawk.lz4#lz4-java;1.11.0 from central in [default]
   com.google.code.findbugs#jsr305;3.0.0 from central in [defau
lt]
   org.apache.commons#commons-pool2;2.13.1 from central in [def
ault]

```

```

org.apache.hadoop#hadoop-client-api;3.5.0 from central in [default]
org.apache.hadoop#hadoop-client-runtime;3.5.0 from central in [default]
org.apache.kafka#kafka-clients;3.9.2 from central in [default]
org.apache.spark#spark-sql-kafka-0-10_2.13;4.2.0 from central in [default]
org.apache.spark#spark-token-provider-kafka-0-10_2.13;4.2.0 from central in [default]
org.scala-lang.modules#scala-parallel-collections_2.13;1.2.0 from central in [default]
org.slf4j#slf4j-api;2.0.17 from central in [default]
org.xerial.snappy#snappy-java;1.1.10.8 from central in [default]

```

facts				modules							arti	
		conf		number		search		dwnlded		evicted		number
dwnlded												

-----		default		11		0		0		0		11
0												

```

:: retrieving :: org.apache.spark#spark-submit-parent-84959c4f-c557-4147-85b2-6a7bac9c1f88
  confs: [default]
  0 artifacts copied, 11 already retrieved (0kB/3ms)
26/10/03 22:29:25 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/10/03 22:29:25 WARN Utils: Service 'SparkUI' could not bind on port 4040. Attempting port 4041.
Spark: 4.2.0
Scala: 2.13.18

```

In [4]: *#using the columns in original dataset to create the explicitly defined columns*

```
Out[4]: ['Date',
         'Time',
         'Booking ID',
         'Booking Status',
         'Customer ID',
         'Vehicle Type',
         'Pickup Location',
         'Drop Location',
         'Avg VTAT',
         'Avg CTAT',
         'Cancelled Rides by Customer',
         'Reason for cancelling by Customer',
         'Cancelled Rides by Driver',
         'Driver Cancellation Reason',
         'Incomplete Rides',
         'Incomplete Rides Reason',
         'Booking Value',
         'Ride Distance',
         'Driver Ratings',
         'Customer Rating',
         'Payment Method']
```

```
In [5]: from pyspark.sql.types import ArrayType, StringType, StructField, S

#same schema as above with the additional eventtimestamp from the p
uber_schema = ArrayType(StructType([
    StructField("Date", StringType(), True),
    StructField("Time", StringType(), True),
    StructField("Booking ID", StringType(), True),
    StructField("Booking Status", StringType(), True),
    StructField("Customer ID", StringType(), True),
    StructField("Vehicle Type", StringType(), True),
    StructField("Pickup Location", StringType(), True),
    StructField("Drop Location", StringType(), True),
    StructField("Avg VTAT", StringType(), True),
    StructField("Avg CTAT", StringType(), True),
    StructField("Cancelled Rides by Customer", StringType(), Tr
    StructField("Reason for cancelling by Customer", StringType
    StructField("Cancelled Rides by Driver", StringType(), True
    StructField("Driver Cancellation Reason", StringType(), Tru
    StructField("Incomplete Rides", StringType(), True),
    StructField("Incomplete Rides Reason", StringType(), True),
    StructField("Booking Value", StringType(), True),
    StructField("Ride Distance", StringType(), True),
    StructField("Driver Ratings", StringType(), True),
    StructField("Customer Rating", StringType(), True),
    StructField("Payment Method", StringType(), True),
    StructField("event_timestamp", StringType(), True)]))
```

```
In [6]: #reading in stream with the kafka source connector to ingest the to
topic = "events"

raw_stream = (spark.readStream.format("kafka").option("kafka.bootst
    .option("startingOffsets", "latest").load())
#checking the schema has the standard kakfa fields
```

```
raw_stream.printSchema()
```

```
root
|-- key: binary (nullable = true)
|-- value: binary (nullable = true)
|-- topic: string (nullable = true)
|-- partition: integer (nullable = true)
|-- offset: long (nullable = true)
|-- timestamp: timestamp (nullable = true)
|-- timestampType: integer (nullable = true)
```

```
In [12]: #parse the incoming JSON payload based on the schema above then expl
parsed_stream = (raw_stream.select(F.from_json(F.col("value")).cast(
uber_stream = (parsed_stream.select(F.explode("records").alias("ube

#additional format preprocessing that is required for stream data,
#replacing the word null with None type
processed_stream = uber_stream.replace("null", None)
#changing numerical features into float types
numeric_cols = ["Avg VTAT","Booking Value","Ride Distance","Custome
for c in numeric_cols:
    processed_stream = processed_stream.withColumn(c,F.col(c).cast(
#create hour column from time column
processed_stream = processed_stream.withColumn("Hour",F.hour("Time"
#covert new eventtimestamp column into actual timestamp type
processed_stream = processed_stream.withColumn("event_timestamp",F.
```

```
In [13]: #calling the models and applying them to the stream data
from pyspark.ml import PipelineModel
from pyspark.ml.regression import DecisionTreeRegressionModel

pipeline_model = PipelineModel.load("models/preprocessing_pipeline"
dt_model = DecisionTreeRegressionModel.load("models/dt_model_final"

feature_stream = pipeline_model.transform(processed_stream)

prediction_stream = dt_model.transform(feature_stream)
```

```
In [ ]: #write prediction results to console sink using writeStream
# completing a test of querying the prediction with streaming data
#query was started before producer.py was run in seperate terminal
prediction_query = (prediction_stream.select("event_timestamp","Dri
    .trigger(processingTime="5 seconds").start())
```

```
26/10/03 22:42:24 WARN ResolveWriteToStream: Temporary checkpoint lo
cation created which is deleted normally when the query didn't fail:
/tmp/temporary-967ff2aa-8c61-4ead-9d77-e27134fdeb05. If it's require
d to delete it under any circumstances, please set spark.sql.streami
ng.forceDeleteTempCheckpointLocation to true. Important to know dele
ting temp checkpoint folder is best effort.
26/10/03 22:42:24 WARN ResolveWriteToStream: spark.sql.adaptive.enab
led is not supported in streaming DataFrames/Datasets and will be di
sabled.
```

Batch: 0

event_timestamp	Driver Ratings	prediction
-----------------	----------------	------------

Batch: 1

event_timestamp	Driver Ratings	prediction
2026-10-03 22:42:29.050667	4.2	4.2264329389131055
2026-10-03 22:42:29.050667	3.4	4.2264329389131055
2026-10-03 22:42:29.050667	NULL	4.239254600618353
2026-10-03 22:42:29.050667	NULL	4.239254600618353
2026-10-03 22:42:29.050667	4.3	4.239254600618353
2026-10-03 22:42:29.050667	NULL	3.7666667302449546
2026-10-03 22:42:29.050667	3.8	4.2264329389131055
2026-10-03 22:42:29.050667	NULL	4.239254600618353
2026-10-03 22:42:29.050667	4.2	4.2264329389131055
2026-10-03 22:42:29.050667	4.5	4.223648643493652
2026-10-03 22:42:29.050667	NULL	4.239254600618353
2026-10-03 22:42:29.050667	4.4	4.239254600618353
2026-10-03 22:42:29.050667	3.7	4.280878855043805
2026-10-03 22:42:29.050667	NULL	4.280878855043805
2026-10-03 22:42:29.050667	5.0	4.280878855043805
2026-10-03 22:42:29.050667	4.7	4.2264329389131055
2026-10-03 22:42:29.050667	NULL	4.239254600618353
2026-10-03 22:42:29.050667	4.2	4.2264329389131055
2026-10-03 22:42:29.050667	NULL	4.239254600618353
2026-10-03 22:42:29.050667	NULL	4.239254600618353

only showing top 20 rows

Batch: 2

event_timestamp	Driver Ratings	prediction
2026-10-03 22:42:34.125754	5.0	4.2264329389131055
2026-10-03 22:42:34.125754	3.3	4.2264329389131055
2026-10-03 22:42:34.125754	NULL	4.239254600618353
2026-10-03 22:42:34.125754	4.9	4.2264329389131055
2026-10-03 22:42:34.125754	4.9	4.239254600618353
2026-10-03 22:42:34.125754	4.5	4.220513768899113
2026-10-03 22:42:34.125754	3.7	4.239254600618353
2026-10-03 22:42:34.125754	NULL	4.239254600618353
2026-10-03 22:42:34.125754	4.4	4.2264329389131055
2026-10-03 22:42:34.125754	4.2	4.2264329389131055
2026-10-03 22:42:34.125754	4.6	4.2264329389131055
2026-10-03 22:42:34.125754	4.3	3.944999986886978
2026-10-03 22:42:34.125754	4.1	4.241025639650149

2026-10-03 22:42:34.125754 4.3	4.2264329389131055
2026-10-03 22:42:34.125754 3.9	4.239254600618353
2026-10-03 22:42:34.125754 NULL	4.239254600618353
2026-10-03 22:42:34.125754 4.1	4.223648643493652
2026-10-03 22:42:34.125754 NULL	4.239254600618353
2026-10-03 22:42:34.125754 3.8	4.2264329389131055
2026-10-03 22:42:34.125754 4.3	4.2264329389131055

only showing top 20 rows

Batch: 3

event_timestamp	Driver Ratings	prediction
2026-10-03 22:42:39.220376 4.2		4.239254600618353
2026-10-03 22:42:39.220376 NULL		4.239254600618353
2026-10-03 22:42:39.220376 NULL		4.239254600618353
2026-10-03 22:42:39.220376 NULL		4.239254600618353
2026-10-03 22:42:39.220376 4.3		4.2264329389131055
2026-10-03 22:42:39.220376 4.6		4.2264329389131055
2026-10-03 22:42:39.220376 NULL		4.280878855043805
2026-10-03 22:42:39.220376 4.3		4.280878855043805
2026-10-03 22:42:39.220376 4.3		4.2264329389131055
2026-10-03 22:42:39.220376 3.8		4.220513768899113
2026-10-03 22:42:39.220376 4.6		4.2264329389131055
2026-10-03 22:42:39.220376 4.3		4.2264329389131055
2026-10-03 22:42:39.220376 4.4		4.2264329389131055
2026-10-03 22:42:39.220376 NULL		4.239090897820213
2026-10-03 22:42:39.220376 4.3		4.241025639650149
2026-10-03 22:42:39.220376 4.1		4.239254600618353
2026-10-03 22:42:39.220376 4.2		4.2264329389131055
2026-10-03 22:42:39.220376 5.0		4.2264329389131055
2026-10-03 22:42:39.220376 4.9		4.220513768899113
2026-10-03 22:42:39.220376 NULL		4.239254600618353

only showing top 20 rows

Batch: 4

event_timestamp	Driver Ratings	prediction
2026-10-03 22:42:44.298787 NULL		4.239254600618353
2026-10-03 22:42:44.298787 3.2		4.2264329389131055
2026-10-03 22:42:44.298787 NULL		4.239254600618353
2026-10-03 22:42:44.298787 NULL		4.239254600618353
2026-10-03 22:42:44.298787 4.0		4.121874988079071
2026-10-03 22:42:44.298787 4.2		4.280878855043805
2026-10-03 22:42:44.298787 NULL		4.239254600618353
2026-10-03 22:42:44.298787 4.2		4.2264329389131055
2026-10-03 22:42:44.298787 4.2		4.223648643493652
2026-10-03 22:42:44.298787 4.3		4.2264329389131055
2026-10-03 22:42:44.298787 NULL		4.239254600618353
2026-10-03 22:42:44.298787 3.2		4.2264329389131055
2026-10-03 22:42:44.298787 3.9		4.2264329389131055

2026-10-03 22:42:44.298787 4.3	4.169162972908188
2026-10-03 22:42:44.298787 4.3	4.239254600618353
2026-10-03 22:42:44.298787 4.3	4.2264329389131055
2026-10-03 22:42:44.298787 NULL	4.239254600618353
2026-10-03 22:42:44.298787 NULL	4.239254600618353
2026-10-03 22:42:44.298787 4.8	4.2264329389131055
2026-10-03 22:42:44.298787 3.7	4.2264329389131055

+-----+-----+-----+

only showing top 20 rows

Batch: 5

event_timestamp	Driver Ratings	prediction	
+-----+-----+-----+			
2026-10-03 22:42:49.38448 4.7		4.239254600618353	
2026-10-03 22:42:49.38448 NULL		4.239254600618353	
2026-10-03 22:42:49.38448 4.8		4.2264329389131055	
2026-10-03 22:42:49.38448 NULL		4.239254600618353	
2026-10-03 22:42:49.38448 NULL		4.364864864864865	
2026-10-03 22:42:49.38448 4.3		4.2264329389131055	
2026-10-03 22:42:49.38448 4.4		4.169162972908188	
2026-10-03 22:42:49.38448 NULL		4.239254600618353	
2026-10-03 22:42:49.38448 NULL		4.239254600618353	
2026-10-03 22:42:49.38448 NULL		4.239254600618353	
2026-10-03 22:42:49.38448 NULL		4.220513768899113	
2026-10-03 22:42:49.38448 NULL		4.239254600618353	
2026-10-03 22:42:49.38448 4.1		4.2264329389131055	
2026-10-03 22:42:49.38448 NULL		4.239254600618353	
2026-10-03 22:42:49.38448 NULL		4.239254600618353	
2026-10-03 22:42:49.38448 4.5		4.239254600618353	
2026-10-03 22:42:49.38448 4.6		4.2264329389131055	
2026-10-03 22:42:49.38448 NULL		4.239254600618353	
2026-10-03 22:42:49.38448 NULL		4.239254600618353	
2026-10-03 22:42:49.38448 4.1		4.2264329389131055	

+-----+-----+-----+

only showing top 20 rows

Batch: 6

event_timestamp	Driver Ratings	prediction	
+-----+-----+-----+			
2026-10-03 22:42:54.47791 NULL		3.7666667302449546	
2026-10-03 22:42:54.47791 NULL		4.2264329389131055	
2026-10-03 22:42:54.47791 NULL		4.239254600618353	
2026-10-03 22:42:54.47791 NULL		4.2264329389131055	
2026-10-03 22:42:54.47791 4.3		4.239254600618353	
2026-10-03 22:42:54.47791 3.1		4.2264329389131055	
2026-10-03 22:42:54.47791 4.4		4.2264329389131055	
2026-10-03 22:42:54.47791 4.6		4.2264329389131055	
2026-10-03 22:42:54.47791 NULL		4.239254600618353	
2026-10-03 22:42:54.47791 NULL		4.239254600618353	
2026-10-03 22:42:54.47791 4.3		4.2264329389131055	
2026-10-03 22:42:54.47791 NULL		4.239254600618353	
2026-10-03 22:42:54.47791 4.2		4.239254600618353	

2026-10-03 22:42:54.47791 NULL	4.239254600618353
2026-10-03 22:42:54.47791 4.4	4.224528294046613
2026-10-03 22:42:54.47791 NULL	4.239254600618353
2026-10-03 22:42:54.47791 3.9	4.239254600618353
2026-10-03 22:42:54.47791 4.2	4.239254600618353
2026-10-03 22:42:54.47791 3.8	4.2264329389131055
2026-10-03 22:42:54.47791 3.7	4.2264329389131055

+-----+-----+

only showing top 20 rows

Batch: 7

event_timestamp	Driver Ratings	prediction	
2026-10-03 22:42:59.554817 4.2		4.311688328202153	
2026-10-03 22:42:59.554817 NULL		4.239254600618353	
2026-10-03 22:42:59.554817 4.7		4.2264329389131055	
2026-10-03 22:42:59.554817 4.5		4.2264329389131055	
2026-10-03 22:42:59.554817 3.4		4.2264329389131055	
2026-10-03 22:42:59.554817 3.0		4.239254600618353	
2026-10-03 22:42:59.554817 3.8		4.2264329389131055	
2026-10-03 22:42:59.554817 5.0		4.2692733459406655	
2026-10-03 22:42:59.554817 4.0		4.2264329389131055	
2026-10-03 22:42:59.554817 NULL		4.239254600618353	
2026-10-03 22:42:59.554817 3.6		4.2264329389131055	
2026-10-03 22:42:59.554817 4.2		4.2264329389131055	
2026-10-03 22:42:59.554817 NULL		4.239254600618353	
2026-10-03 22:42:59.554817 NULL		4.239254600618353	
2026-10-03 22:42:59.554817 4.2		4.239254600618353	
2026-10-03 22:42:59.554817 NULL		4.239254600618353	
2026-10-03 22:42:59.554817 4.9		4.2264329389131055	
2026-10-03 22:42:59.554817 4.9		4.2264329389131055	
2026-10-03 22:42:59.554817 4.3		4.224528294046613	
2026-10-03 22:42:59.554817 NULL		4.239254600618353	

+-----+-----+

only showing top 20 rows

Batch: 8

event_timestamp	Driver Ratings	prediction	
2026-10-03 22:43:04.63365 NULL		4.239254600618353	
2026-10-03 22:43:04.63365 4.6		4.2264329389131055	
2026-10-03 22:43:04.63365 NULL		4.239254600618353	
2026-10-03 22:43:04.63365 NULL		4.239254600618353	
2026-10-03 22:43:04.63365 NULL		4.239254600618353	
2026-10-03 22:43:04.63365 NULL		4.239254600618353	
2026-10-03 22:43:04.63365 4.0		4.2264329389131055	
2026-10-03 22:43:04.63365 4.2		4.2264329389131055	
2026-10-03 22:43:04.63365 NULL		4.239254600618353	
2026-10-03 22:43:04.63365 4.6		4.2264329389131055	
2026-10-03 22:43:04.63365 4.2		4.2264329389131055	
2026-10-03 22:43:04.63365 3.2		4.2264329389131055	

2026-10-03 22:43:04.63365 NULL	4.239254600618353
2026-10-03 22:43:04.63365 4.3	4.2264329389131055
2026-10-03 22:43:04.63365 4.6	4.2692733459406655
2026-10-03 22:43:04.63365 NULL	4.239254600618353
2026-10-03 22:43:04.63365 3.8	4.239254600618353
2026-10-03 22:43:04.63365 4.5	4.2264329389131055
2026-10-03 22:43:04.63365 4.6	4.239254600618353
2026-10-03 22:43:04.63365 4.7	4.2264329389131055

+-----+-----+

only showing top 20 rows

Batch: 9

event_timestamp	Driver Ratings	prediction
2026-10-03 22:43:09.714755 4.7		4.2264329389131055
2026-10-03 22:43:09.714755 4.7		4.239254600618353
2026-10-03 22:43:09.714755 3.2		4.239254600618353
2026-10-03 22:43:09.714755 4.6		4.126886778844978
2026-10-03 22:43:09.714755 4.2		4.239254600618353
2026-10-03 22:43:09.714755 4.6		4.2264329389131055
2026-10-03 22:43:09.714755 4.7		4.2264329389131055
2026-10-03 22:43:09.714755 4.2		4.2264329389131055
2026-10-03 22:43:09.714755 4.7		4.246002176113484
2026-10-03 22:43:09.714755 NULL		4.246002176113484
2026-10-03 22:43:09.714755 3.5		4.239254600618353
2026-10-03 22:43:09.714755 4.9		4.169162972908188
2026-10-03 22:43:09.714755 4.2		4.169162972908188
2026-10-03 22:43:09.714755 NULL		4.239254600618353
2026-10-03 22:43:09.714755 NULL		4.239254600618353
2026-10-03 22:43:09.714755 4.3		4.2264329389131055
2026-10-03 22:43:09.714755 4.4		4.223648643493652
2026-10-03 22:43:09.714755 NULL		4.239254600618353
2026-10-03 22:43:09.714755 4.3		4.223648643493652
2026-10-03 22:43:09.714755 NULL		4.239254600618353

+-----+-----+

only showing top 20 rows

Batch: 10

event_timestamp	Driver Ratings	prediction
2026-10-03 22:43:14.859952 4.2		4.239254600618353
2026-10-03 22:43:14.859952 4.8		4.239254600618353
2026-10-03 22:43:14.859952 4.4		4.2264329389131055
2026-10-03 22:43:14.859952 NULL		4.239254600618353
2026-10-03 22:43:14.859952 4.4		4.239254600618353
2026-10-03 22:43:14.859952 3.8		4.2264329389131055
2026-10-03 22:43:14.859952 4.6		4.239254600618353
2026-10-03 22:43:14.859952 3.3		4.2264329389131055
2026-10-03 22:43:14.859952 NULL		4.239254600618353
2026-10-03 22:43:14.859952 4.1		4.239254600618353
2026-10-03 22:43:14.859952 4.9		4.239254600618353
2026-10-03 22:43:14.859952 4.9		4.2264329389131055

2026-10-03 22:43:14.859952	4.3	4.223648643493652
2026-10-03 22:43:14.859952	4.2	4.239254600618353
2026-10-03 22:43:14.859952	4.6	4.2692733459406655
2026-10-03 22:43:14.859952	4.3	4.2264329389131055
2026-10-03 22:43:14.859952	NULL	4.239254600618353
2026-10-03 22:43:14.859952	4.6	4.2264329389131055
2026-10-03 22:43:14.859952	4.5	4.2264329389131055
2026-10-03 22:43:14.859952	NULL	4.2264329389131055

only showing top 20 rows

```
In [ ]: #stopping query and confirming that the query is actually stopped
prediction_query.stop()
print(prediction_query.isActive)
```

```
26/10/03 22:43:16 WARN DAGScheduler: Failed to cancel job group fbdd
cfef-dd85-4732-a25f-3128900a0fed. Cannot find active jobs for it.
26/10/03 22:43:16 WARN DAGScheduler: Failed to cancel job group fbdd
cfef-dd85-4732-a25f-3128900a0fed. Cannot find active jobs for it.
```

1c. Window Aggregation

```
In [ ]: #window aggregation used - average predicted value per window

# window size every 30 seconds (6 batches)
# slide interval 15 seconds wide (3 batches), so overlap between wi
# watermark every 10 seconds (2 batches)

windowed_predictions = (prediction_stream.withWatermark("event_time",
    ("event_timestamp", "30 seconds", "15 seconds")).agg(F.avg("predi
    F.count("*").alias("prediction_count")))

#window query to diaply results to the console to demonstrate real-
window_query = (windowed_predictions.select(F.col("window.start").a
    F.round("avg_predicted_driver_rating", 3).alias("avg_predic
    .writeStream.outputMode("update").format("console").option("tru
    .start())
```

```
26/10/03 23:09:30 WARN ResolveWriteToStream: Temporary checkpoint lo
cation created which is deleted normally when the query didn't fail:
/tmp/temporary-f16e50e4-64e8-4c76-80d0-44989d44fbb4. If it's require
d to delete it under any circumstances, please set spark.sql.streami
ng.forceDeleteTempCheckpointLocation to true. Important to know dele
ting temp checkpoint folder is best effort.
26/10/03 23:09:30 WARN ResolveWriteToStream: spark.sql.adaptive.enab
led is not supported in streaming DataFrames/Datasets and will be di
sabled.
26/10/03 23:09:30 WARN MicroBatchExecution: Disabling AQE since AQE
is not supported in stateful workloads.
```

Batch: 0			
window_start	window_end	avg_predicted_driver_rating	prediction_count

Batch: 1			
window_start	window_end	avg_predicted_driver_rating	prediction_count
2026-10-03 23:09:30	2026-10-03 23:10:00	4.231	800
2026-10-03 23:09:15	2026-10-03 23:09:45	4.231	800

Batch: 2			
window_start	window_end	avg_predicted_driver_rating	prediction_count
2026-10-03 23:09:30	2026-10-03 23:10:00	4.232	1600
2026-10-03 23:09:45	2026-10-03 23:10:15	4.233	800

Batch: 3

window_start	window_end	avg_predicted_driver_rating
prediction_count		
2026-10-03 23:09:30	2026-10-03 23:10:00	4.231
2400		
2026-10-03 23:09:45	2026-10-03 23:10:15	4.231
1600		

Batch: 4

window_start	window_end	avg_predicted_driver_rating
prediction_count		
2026-10-03 23:09:30	2026-10-03 23:10:00	4.232
3200		
2026-10-03 23:09:45	2026-10-03 23:10:15	4.232
2400		

Batch: 5

window_start	window_end	avg_predicted_driver_rating
prediction_count		
2026-10-03 23:09:45	2026-10-03 23:10:15	4.231
3200		
2026-10-03 23:10:00	2026-10-03 23:10:30	4.229
800		

Batch: 6

window_start	window_end	avg_predicted_driver_rating
prediction_count		
2026-10-03 23:09:45	2026-10-03 23:10:15	4.232
4000		
2026-10-03 23:10:00	2026-10-03 23:10:30	4.231
1600		

Batch: 7

window_start	window_end	avg_predicted_driver_rating
prediction_count		
2026-10-03 23:09:45	2026-10-03 23:10:15	4.231
4800		
2026-10-03 23:10:00	2026-10-03 23:10:30	4.231
2400		

Batch: 8

window_start	window_end	avg_predicted_driver_rating
prediction_count		
2026-10-03 23:10:00	2026-10-03 23:10:30	4.231
3200		
2026-10-03 23:10:15	2026-10-03 23:10:45	4.231
800		

Batch: 9

window_start	prediction_count	window_end	avg_predicted_driver_rating
2026-10-03 23:10:00	4000	2026-10-03 23:10:30	4.23
2026-10-03 23:10:15	1600	2026-10-03 23:10:45	4.229

Batch: 10

window_start	prediction_count	window_end	avg_predicted_driver_rating
2026-10-03 23:10:00	4800	2026-10-03 23:10:30	4.23
2026-10-03 23:10:15	2400	2026-10-03 23:10:45	4.23

Batch: 11

window_start	prediction_count	window_end	avg_predicted_driver_rating
2026-10-03 23:10:30	800	2026-10-03 23:11:00	4.231
2026-10-03 23:10:15	3200	2026-10-03 23:10:45	4.23

```
-----  
Batch: 12  
-----
```

```
+-----+-----+-----+  
+-----+  
|window_start      |window_end      |avg_predicted_driver_rating  
|prediction_count|  
+-----+-----+-----+  
+-----+  
|2026-10-03 23:10:30|2026-10-03 23:11:00|4.231  
|1600              |  
|2026-10-03 23:10:15|2026-10-03 23:10:45|4.23  
|4000              |  
+-----+-----+-----+  
+-----+
```

```
In [21]: window_query.stop()
```

```
26/10/03 23:10:44 WARN DAGScheduler: Failed to cancel job group 8c01  
66cf-e473-4ece-9327-356e59aee5fb. Cannot find active jobs for it.  
26/10/03 23:10:44 WARN DAGScheduler: Failed to cancel job group 8c01  
66cf-e473-4ece-9327-356e59aee5fb. Cannot find active jobs for it.
```

The above real-time trend monitoring shows the average predicted driver rating as well as the number of records used for this prediction over a specific window. Due to the choice of a 30 second window with a 5 second delay between batches, the same window is shown multiple times with increasing number of records. For example, the |2026-10-03 23:10:00|2026-10-03 23:10:30 starts in batch 5 then is finally shown in batch 10 with 4800 records or 6 batches with a 5 second delay in between. It can also be seen the average rating slightly changing over that time as more records are included but stabilising. There is an overlap between windows of approximately 15 seconds (sliding interval), which is used to smooth out the data and can be seen from the similar average values between the windows. Finally a 9 second watermark interval is used to catch any late incoming data. This time makes sense as the batches are sent every 5 seconds, so the 9 seconds would be used to catch any late coming data before the next batch. For this streaming simulation, the time between batches is very accurate but for real life streaming, watermark would be a lot more active in the query.

5. Reflection Section

1. Why was the model selected?

The model chosen was the decision tree with default settings. As mentioned in Part A, it was discovered that there was a very low correlation between all numerical values and Driver Ratings. Additionally, reviewing the importance of all features and creating a dataset with only those values didn't produce a better model. As the ratings was a numerical value, regression models were

investigated with RMSE, R2 and MAE used to evaluate. Since these values changed very little, the decision tree was chosen as the R2 was acceptable and the RMSE and MAE matched to other models. In the future, potentially a classification model would be able to aid in predicting ratings, if they were translated into rating range categories rather than straight numerical values. Alternatively, if additional data or features were available, they could potentially aid in creating a more ideal model.

2. Why was this window configuration selected?

The configuration model chosen (30 window, 15 second interval, 9 second watermark) was chosen due to the knowledge of the data as well as the established 5 second producer batch interval of 5 seconds. The 30 second window provides adequate smoothing of the data, especially when reviewing the number of NULL entries for driver rating, giving an average over 6 batches with a decent amount of data. This would smooth out any spikes in the data that wouldn't reflect the overall average of the rating. The overlap between windows of 15 seconds is a middle ground between keeping existing data, 50% of the previous window, and introducing new data. This would be helpful to monitor temporal changes like demand or satisfaction, especially in terms of driver ratings. Finally, the watermark timing is a good compromise with a 5 second batch interval as this isn't retaining data for too long but sensible enough to capture any late arriving data.

3. What challenges arose during streaming?

The main issue I had was ensuring my code functioned properly and was able to read in data from proper locations. Since I was using VS Code for this assessment, Part A was completed without any issue, using a python kernel and committing to github as this environment I had used many times before. When setting up Docker containers, ensuring my producer code was running properly with logs saved to a location that I could read back in, and ensuring Kafka was running properly in my jupyter notebook, this was the biggest issue I experienced. I had to ensure I was in the proper location before saving my pipeline, ensure my containers (PySpark, Zookeeper and Kafka) spoke to each other, and create a separate spark environment specifically with Kafka included. I had to ensure I was in the correct container and directory with a separate VS Code window before running the producer code. From my experience, the biggest challenge comes from initial setup of the environment and ensuring all containers and codes can communicate.

4. How would the solution behave with higher volume of data?

The biggest issue with larger volume of data would be the preprocessing and modelling completed with each batch. Comparatively, there are a lot of steps

within the Spark code before a prediction can be outputted and with a batch arriving every 5 seconds, this could lead to a bottleneck with a higher volume of data. If the Spark processing time is greater than 5 seconds, a backlog could be created and the system would lose its real-time monitoring. A potential solution to use parallel computing, within the producer and consumer. The Kafka topics created in the producer could be partitioned, with each partition having its own Spark processing so the larger volume of streamed data is separated before arrival. Then in the consumer with Spark a stream-stream join would be required using the watermark interval as well as a temporal constraint, which could come from the eventtimestamp.